Ruprecht-Karls-Universität Heidelberg
Institute of Computer Science
Artificial Intelligence for Programming (AIP)

Master's Thesis

TITLE

Name:	Niklas Loeser
Matrikelnummer:	3737339
Betreuer:	Prof. Dr. Artur Andrzejak
Datum der Abgabe	2024-09-30

I hereby certify that I have written the work myself and that I have not used any sources or aids other than those specified and that I have marked what has been taken over from other people's works, either verbatim or in terms of content, as foreign. I also certify that the electronic version of my thesis transmitted completely corresponds in content and wording to the printed version. I agree that this electronic version is being checked for plagiarism at the university using plagiarism software.

Heidelberg, den 2024-06-11

_____

Niklas Loeser

Abstract

- English -

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut.

Abstract

- German -

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut.

Contents

1 Introduction I
1.1 Motivation I
1.2 Goals I
1.3 Contributions I

2 Background & Related Work 3
2.1 Causal Language Models 3
2.2 Text Generation Strategies 5
2.3 Meta-Strategies 6
2.4 Guided Generation 6
2.5 Transformers Library 7
2.6 Code Generation 7
2.7 Language Server Protocol 7
2.8 Related Work 11

3 Design 12
3.1 Interrupt-based meta-strategy 12
3.2 Language Server Protocol aided generation 12
3.3 Dataset design 12

4 Implementation 13
4.1 Interrupt-based meta-strategy 13
4.2 Language Server Protocol aided generation 13
4.3 Dataset design 13

5 Results 14
5.1 Collection method 14
5.2 Possible configurations 14
5.3 Results 14

6 Evaluation 15
6.1 Research Questions 15
6.2 Evaluation difficulties 15
6.3 Evaluation 15

7 Conclusion 16
7.1 Recap 16
7.2 Outlook 16

I Bibliography 17

II List of Figures 18

III List of Tables 19

IV List of Code 20

V List of Examples 21

1 Introduction

Code generation describes the task of generating code for a specific programming language. Generally the code could be generated from a description of the desired functionality, or as a continuation of existing code.

An important use of code generation is to provide code completions in an Integrated Development Environment (IDE). There the completed code removes the need to write boilerplate code.

1.1 Motivation

- Problems:
  - Outdated knowledge and retraining not possible
    - Not flexible enough, as cannot find all needed knowledge for all domains
    - Fine-tuning / retraining slow
  - Dynamic techniques can inject up-to-date information, but are:
    - Relevance: Not specific enough, as they cannot find relevant information
      - Model confusion, as too many nonrelevant tokens are injected and context window is limited
    - Speed:
      - Reliance to terminate, as some iterative methods do not converge
      - Total generated tokens
    - Are not localized, as the model does not know **where** the injected knowledge applies to

1.2 Goals

Goal: Technique to provide the model with up-to-date knowledge while answering the following research questions:

(Reliance over Quality, as quality output does not matter if the fcking program does not terminate)

1. RQ1: How to find relevant information needed for the model generation
2. RQ2: How to provide the model with the relevant information to nudge it towards the desired outcome (understeering causes no change, oversteering is too inflexible, we want nudges)
3. RQ3: How to decrease the performance impact upon the default generation and ensure a reliable generation
4. RQ1: How to find up-to-date information on local code and dependencies
5. RQ2: How to determine the relevancy of the found information on local code and dependencies
6. RQ3:

1.3 Contributions

- General interrupt-based meta-strategy
  - Almost no performance penalty
  - Can provide localized information on-the-fly
  - Robust as it:

  - Gently guides the generation of the model
  - Nudges the model by providing injected context
  - Reduces interrupts
- Implementation of interrupt-based meta-strategy on code generation
  - Programming language independence using LSP
- Creation of custom dataset to evaluate functional and **approach** correctness
  - Automated
  - Plotting
  - Stats collection => Excel
  - Includes pinned dependencies and Python version
  - Implements security concept using Docker

2 Background & Related Work

This chapter provides important background information and related work, necessary to understand the main contributions of this thesis. Firstly, Causal Language Modelss (CLMs) are introduced in Section 2.1. Thereafter, Section 2.2 discusses commonly used text generation strategies.

The sections Section 2.3 and Section 2.4 showcase strategies to modify the text generation of a fixed pretrained CLM. Section 2.5 displays important components of the popular Hugging Face libary, which enables CLM workflows using all of the techniques introduced in the sections before.

Thereafter, Section 2.6 highlights important aspects of code generation. Continuing, Section 2.7 introduces the functionality of the Language Server Protocol (LSP). At last, Section 2.8 introduces previous work.

2.1 Causal Language Models

This section will first introduce the concept of a Language Models (LMs), then move from neural language models, to CLMs. Important text generation strategies for CLM are discussed in depth in the next section.

Language Models

Generally, LMs are probabilistic models of natural language[1]. They predict the next upcoming words based on already seen words[1]:

Previous text	*"The quick brown fox jumps over the lazy _"*
Prediction	*"dog"*

Example 1: Language model predicting the last word of the famous English pangram.

Development on LMs has led to powerful models. These now find widespread use in various applications, such as ChatGPT[2].

Neural Language Models

Neural language models are a type of LM. Their name stems from the McCulloch-Pitts neuron, which is a simplified view of a biological neuron as a computational unit[3,1].

In contrast, a modern neural language model is a network of small computing units. Each of the units is a function that takes a sequence of values and returns a singular output value[1]:

$$f_{\text{unit}} : \mathbb{R}^n \to \mathbb{R}$$

Figure 1: Function domain of a singular computing unit.

Combining many units into a network allows the computation of a complex function f_{network}. The behavior of f_{network} can be changed by modifying the parameters inside of the network. By applying changes according to feedback, complex functions can be learned.

One very popular architecture for neural language models is the Transformers architecture.

As can be seen in Figure 1, the network is unable to process sequences of natural language directly. The text must first be encoded into numbers, the model can work with. After processing the encoded text, the result can be decoded into natural language once again.

Each neural language model encodes natural language into increasing numbers called tokens using a vocabulary V. The vocabulary is usually fixed, but can be changed as will

be shown in the thesis. Each token represents a piece of natural language, such as a part of a word, or even parts of a sentence. The models initially map natural language to tokens for processing. After processing the tokens these are decoded back into natural language.

Vocabulary	*{"The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog", ".", "<BOS>", "<EOS>"}*
Text	*"The quick brown fox jumps"*
Tokens	*{10, 0, 1, 2, 3, 4}*
Text fragments	*{"<BOS>", "The", "quick", "brown", "fox", "jumps"}*

Example 2: Natural language encoded into tokens.

Example 2 showcases the tokenization for the start of the famous English pangram. The tokens values correspond to the index in the given vocabulary. The given vocabulary only provides tokens for full words and does not provide tokens for sub-word fragments. Not only does the vocabulary contain tokens for natural language, but also special tokens. These special tokens encode further semantic information for the CLM. Common special tokens are the Begin of Sequence (BOS), End of Sequence (EOS) and Padding (PAD) tokens:

BOS Marks the beginning of a natural text. Used to delimit several unrelated texts.

EOS Marks the end of a natural text. Used in combination with BOS. The text generation is stopped, when the model predicts the EOS token as the next token.

PAD Used to pad sequences from either side to a fixed length. This is necessary for the model to process several sequences of varying lengths at once.

Definition

Causal language models in turn are a type of neural language model. They rely on the assumption that the probability of a word occuring in a document depends only on the words occuring before it.

This property is called autoregressivity and highly favourable towards text generation, as it enables the continuation of any text. Given a text i_0 as input, the causal language model can predict the next word, as all previous words i_0 are known. After predicting the next word, the word is appended to the input to form the new input i_1. This process is repeated until the model predicts the end of the document using the special EOS token after n steps. i_n then forms the output and the whole generated text.

When applied to text completion or code completion, a program need only return the output i_n without the prefix i_0.

For the purposes of this thesis, a CLM can be defined using the following components:

- Raw next token prediction function f_{rntpf}
- Text generation strategy
- Autoregressive property

The raw next token prediction function is a mathematical function that takes a sequence of tokens and returns the prediction for the next tokens:

$$f_{\text{rntpf}}(x) = u$$
$$f_{\text{rntpf}} : \mathbb{N}^n \to \mathbb{R}^{|V|}$$

Figure 2: Raw next token prediction function.

The output u has the shape of the vocabulary, where each value u_i represents the logistical probability of token i being the next token. These values are called logits.

In an additional step, the logistical probabilities are normalized into values in the range of $(0, 1)$ to form a probability distribution.

The intermediary step of forming logits provides the benefit, that individual logits can be changed individually without disrupting the probability distribution. This property is essential for the main contributions in this thesis.

2.2 Text Generation Strategies

To generate a full sequence of tokens and thus a document, the model must choose the next probable token in the document using a text generation strategy TGS. After choosing the next token $\hat{w}$, where $1 \leq \hat{w} \leq |\text{vocab}|$, it is appended to the old input x to form x':

$$x' = x^\frown\langle\hat{w}\rangle = (x_1, x_2, ..., x_n, \hat{w})$$

. Depending on the text generation strategy, the models generate different documents of varying quality. Each text generation strategy has different advantages and drawbacks. The next sections will introduce a few core sampling strategies and highlight the main advantages and drawbacks.

Greedy Decoding

Greedy decoding is the simplest of sampling strategies. The strategy always chooses the token with highest probability. This always yields the current best next token, but not necessarily the best token overall.

Its main advantage is the simplicity. The strategy does not require additional state. Only the current probabilities are needed. This also leads to low hardware requirements, compared to other strategies.

As the current best next token is not often the best token overall, the strategy does not produce the highest quality documents.

$$\hat{w} = \text{argmax}_{w \in V} P(w \mid w_{<n})$$

Figure 3: Greedy Decoding

Beam Search

Greedy decoding suffers from the problem of always choosing the current most probable token, even if it is not the best token overall. Beam search improves upon this by modeling the decoding process as a search tree. Instead of choosing only a single token, the strategy keeps k possible token choices. These k possible token choices are called hypothesis. At each time step the beam search uses the current k hypothesis to independently continue the generation to create $k * V$ possible token combinations. The strategy then keeps the top-k combinations as new hypothesis.

By modeling the decoding process as a search tree, beam search can choose tokens that are better overall. This leads to a higher quality generation.

But as beam search keeps k hypothesis and decodes $k * V$ possible token combinations, the strategy requires more memory and processing power.

Top-k

Top-k is a generalization of greedy decoding and a sampling strategy. Sampling strategies randomly choose the next token from the probability distribution. Top-k chooses the next token from the top k most probable next tokens. Hence, the model chooses a wider variety of tokens, leading to richer outputs.

As top-k is a generalization of greedy decoding, it has the advantage of being more powerful.

The main drawback is the loss of determinism. Lacking determinism may complicate interpretability methods.

Top-p

Top-k always chooses from the top k most probable tokens, even if most tokens are very unlikely and only few of the k tokens are probable. Top-p allows for more dynamic shapes of probability distributions, by only sampling from the top p percent of tokens.

Temperature sampling

Temperature sampling takes a different approach to top-p. It does not adapt to the shape of the probability distribution, but reshapes it. It divides the raw probability scores by a temperature τ:

$$y = \text{softmax}\left(\frac{u}{\tau}\right)$$

Figure 4: Temperature Sampling

The softmax function normalizes the logits to the range $(0, 1)$, where the sum of all values equals 1. When dividing by $\tau < 1$ all values increase in total. After normalization this skews the probability distribution towards the most probable tokens. In contrast, when dividing by $\tau > 1$ the probability distribution is flattened. This leads to a more uniform distribution.

2.3 Meta-Strategies

CLMs provide many different ways to tune the generated text. From switching the model itself, to choosing a different text generation strategy, many different combinations are possible. Sometimes these knobs are not enough and an integration to external tooling is needed. Integrating external tooling into the model generation is harder, the deeper the integration should be rooted. The simplest form of integration is to reverse orders and integrated the model into a wider meta-strategy. This chapter will introduce common meta-strategies and discuss drawbacks and benefits.

A meta-strategy is a strategy that incorporates the model text generation into a wider strategy. The meta-strategies treat the models as a black box that produces a generated text, given an input. The meta-strategies may use this black box indefinitely. In addition, several black boxes could be combined.

Generally, the main benefits are ease of use and flexibility. Any model can easily be integrated into the meta-strategy and stay interchangable. By combining several models into a multi-turn process, including external tooling, powerful behavior is possible.

Models do not have up-to-date information and are sometimes unable to solve problems on their own. Meta-strategies can remedy these issues, by providing the model with up-to-date information and focusing it towards a goal.

Common drawbacks of meta-strategies are the added complexity towards the already complex text generation. Also calling models several times makes the process slower.

Chain of Thought

[4]

Tree of Thought

2.4 Guided Generation

2.5 Transformers Library

2.6 Code Generation

Code generation is a task in the general domain of text generation. It refers to the generation of any type of source code. This section will first introduce several important applications for code generation. Then it will list several problems common to specifically code generation. At last general approaches to improve code generation are shown.

Uses

This section will introduce three common use cases, which slightly differ in their requirements. Common requirements are the following:

- The generated code should be syntactically correct.
- The generated code should be semantically correct.
- To increase maintainability, the generated code should follow best practices.
- The developer might be working on a single file or a large project.
- The developer could work on a project of **any** programming language.
- Knowledge about related code:

The CLM performing the task should take the following context into consideration when generating code: At the core, the CLM has the context of the local file that is edited. The next wider context is code in a local repository, when working on a larger project. At last, the model should use knowledge of installed packages.

The first use case is **code completion** for use in IDEs. In this scenario, a developer will have installed tooling to run autocomplete in parallel to his typing. To display the autocompletion before the developer continues writing, the code must be generated in a timely fashion.

The second use case is for generation of **unit tests**. This scenario differs in the size of the generated code. Generated unit tests generally are larger than code generated as inline code completion. Additionally, the generated code

The last use case is for prototyping, in which a single or several files are generated. The difference to other use cases is the increased runtime. Moreover, the code is generated from a task the developer provided.

Problems

Current code generation models and strategies are capable of the following:

- Generating syntactically correct code.
- Generating semantically correct code.
- Generating code for any programming language.

Current techniques still struggle generating code, which is aware of the local code base.

Requirements, which current techniques do not fulfill are topics, such as following best practices and having knowledge about related code.

General approaches

2.7 Language Server Protocol

Many modern editors and IDEs provide features such as:

- Autocompletion
- Signature help
- Go to the definition of a symbol

- Go to the uses of a symbol
- Provide diagnostics data on the current file
- Rename symbols

This list is not exhaustive, and the features between different editors may wary. These features are vital for any programmer, as the effort to create and maintain code is significantly reduced. Navigating large code bases and finding documentation on used code is highly beneficial.

These many features take a significant effort to implement for a **single** programming language and editor. Providing support for several programming languages massively increases the implementation effort. In addition, the created features must be maintained next to the editor itself. As more programming languages are designed and are adopted, the need for tooling for these programming languages arises. Editor maintainers must then integrate these features for the new programming language.

Providing support for an ever increasing amount of programming languages is hard, but supporting these features for an ever increasing amount of programming languages in every editor is impossible. Many diverse editors exist, such as Emacs, VSCode, Neovim (modern fork of Vim), Zed (the successor of Atom), Jetbrains IDEs, Helix and many more.

Thus, a solution is needed to extract thes **common** features from the editors and provide them centrally, such that every editor need only implement an interface to integrate the common features. The Language Server Protocol is an effort to standardize a common feature set for programming language tooling for use in IDEs. The protocol was designed my Microsoft for use in its Visual Studio Code editor, but is now an open standard. This thesis will focus on version 3.17 of the protocol. The protocol includes many different features, of which the following sections introduce a few important. The full specification may be accessed

Design

The protocol is based on a bidirectional client-server architecture. The client is a tool such as the IDE. The server, henceforth called the language server, is programming language specific tooling that provides a subset of the protocol. Both the client and server must not implement the full protocol. The protocol is capability based. Both parties define and communicate their capabilities, then find common capabilities and restrict their use to them. Each client can connect to multiple language servers, and each language server can serve multiple clients. As such a client could use a language server for the Rust programming language while the user is coding on a Rust project. As soon as the user switches projects to a web project, the editor could then connect to several language servers to provide support for HTML, CSS and JavaScript at once. In that scenario several simultaneous language servers are necessary, as CSS and JavaScript can be embedded into HTML. The communication between the client and server must be bidirectional, as the server must be able to notify the client about events occuring, such as new diagnostic information.

JSON-RPC

The base protocol relies on JavaScript Object Notation-Remote Procedure Call (JSON-RPC). The following section will give a short introduction on the protocol. The content part is specified using JSON-RPC. JSON-RPC is a light-weight and stateless protocol to encode Remote Procedure Calls (RPCs). The specification defines data structures and processing rules. As such, the protocol is transport-agnostic and can be used with any type of message passing framework. The data structures are encoded using the JavaScript Object Notation (JSON) data format.

The protocol is versioned. Currently the versions 1.0 and 2.0 are specified. The LSP uses version 2.0 of JSON-RPC. Henceforth, the sections will focus on version 2.0 of the protocol.

JSON-RPC defines three types of messages:

- Requests
- Responses
- Notifications

, though notifications could be considered a special case of requests. The following section will provide details on each type.

Messages will be defined using TypeScript interfaces, in accordance with the LSP specification. A general message has the following interface:

```typescript
interface Message {
    jsonrpc: string;
}
```

Listing 1: General message interface

To define the other message types, additional TypeScript types are introduced:

- integer: A signed 32-bit integer aliased to the TypeScript type number.
- uinteger: An unsigned 32-bit integer aliased to the TypeScript type number.
- LSPArray:

Request objects have the following JSON format, as defined by the following json format:

```json
{
    "jsonrpc": "string",
    "method": "string",
    "params": "any",
    "id": "string|number|null"
}
```

Listing 2: Request object format

The jsonrpc field contains the version used and must be set to 2.0. The method field contains the name of the remote procedure to be called. Any parameters needed to call the remote function should be passed in the optional field params. At last, the id field identifies the message and thus allows correlation between requests and responses. Using fractional numbers is discouraged, as the floating point accuracy issue contradicts the identification of the id.

Each request must be answered by a response. The response object has the following format:

```json
{
    "jsonrpc": "string",
    "result": "any",
    "error": {
        "code": "number",
        "message": "string",
        "data": "any"
    },
    "id": "string|number|null"
}
```

Similar to the request object, the jsonrpc field must be set to 2.0. Either the result or error field will be set, depending on the success of the request. To correlate requests and responses, the response will contain the id of the request in the field id.

Error responses contain the error code, a short descriptive message and optional additional data. The specification defines the following error codes and messages:

Code	Message	Meaning
−32700	Parse error	The server received invalid JSON
−32600	Invalid Request	The request is not a valid request object
−32601	Method not found	The method does not exist or is not available
−32602	Invalid params	Invalid method parameters
−32603	Internal error	An internal error occurred while processing the request
−32000	Server error	Reserved for implementation-defined server errors

In addition, the protocol allows custom error codes.

The final type of message is the notification. Notifications are a special type of request, as they do not contain an id. Notifications do not require a response from the server.

Base protocol

The base protocol defines the general format, in which messages are sent between the client and server. Each message contains a headers and a content part, separated by \r\n.

The headers part contains at least one header field, which are key-value pairs separated by a colon. Each header field is terminated by a \r\n. The following headers are currently supported:

- Content-Length: A mandatory header that specifies the length of the content part in bytes.
- Content-Type: The Multipurpose Internet Mail Extensions (MIME) type of the content part. The default is application/vscode-jsonrpc; charset=utf-8 and no other charset is currently supported.

The content part is a JSON object, as defined by the JSON-RPC specification.

Basic JSON structures

Lifecycle Messages

Document Synchronization

Language Features

Completion

- types of items (functions, fields, etc.)
- documentation
- flags, such as deprecated
- implementation up to servers
- many servers for many languages

Signature help

- Arguments

Note thingy

2.8 Related Work

3 Design

3.1 Interrupt-based meta-strategy

3.2 Language Server Protocol aided generation

3.3 Dataset design

4 Implementation

4.1 Interrupt-based meta-strategy

4.2 Language Server Protocol aided generation

4.3 Dataset design

5 Results

5.1 Collection method

5.2 Possible configurations

5.3 Results

6 Evaluation

6.1 Research Questions

6.2 Evaluation difficulties

6.3 Evaluation

7 Conclusion

7.1 Recap

7.2 Outlook

I Bibliography

[1] "Speech and Language Processing." Accessed: Jun. 04, 2024. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/

[2] "ChatGPT." Accessed: Jun. 10, 2024. [Online]. Available: https://chatgpt.com/

[3] "https://home.csulb.edu/~cwallis/382/readings/482/mccolloch.logical.calculus.ideas.1943.pdf." Accessed: Jun. 10, 2024. [Online]. Available: https://home.csulb.edu/~cwallis/382/readings/482/mccolloch.logical.calculus.ideas.1943.pdf

[4] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." Accessed: Oct. 10, 2023. [Online]. Available: http://arxiv.org/abs/2201.11903

II List of Figures

Figure 1: Function domain of a singular computing unit. 3

Figure 2: Raw next token prediction function. 4

Figure 3: Greedy Decoding 5

Figure 4: Temperature Sampling 6

III List of Tables

IV List of Code

Listing 1: General message interface 9

Listing 2: Request object format 9

V List of Examples

Example 1: Language model predicting the last word of the famous English pangram. 3

Example 2: Natural language encoded into tokens. 4

VI Acronyms

BOS: Begin of Sequence

BPE: Byte Pair Encoding

CLM: Causal Language Models

Code LLM: Large Language Model for Code

EOS: End of Sequence

IDE: Integrated Development Environment

JSON: JavaScript Object Notation

JSON-RPC: JavaScript Object Notation-Remote Procedure Call

LLM: Large Language Models

LM: Language Model

LSP: Language Server Protocol

MIME: Multipurpose Internet Mail Extensions

PAD: Padding

RAG: Retrieval Augmented Generation

RPC: Remote Procedure Call

QR code



Scan the QR code to access the source code of this thesis:
https://github.com/data-niklas/master_thesis